

Селекторы, сравнение селекторов, **CSS** переменные

Работу выполнили
Румянцева Анастасия,
Чернятьева Варвара,
ИБ-31 БО

Селекторы представляют **структуру веб страницы**. С их помощью создаются правила для форматирования элементов веб-страницы. Селекторами могут быть элементы, их классы и идентификаторы, а также псевдоклассы и псевдоэлементы.

Универсальный селектор

***** - универсальный селектор - применяет заданные свойства для всех элементов страницы

Соответствует любому HTML-элементу. Например,

```
* {  
    margin: 0;  
}
```

обнулит внешние отступы для всех элементов сайта. Также селектор может использоваться в комбинации с псевдоклассом или псевдоэлементом:

```
*:after {  
    CSS-стили  
}
```

```
*:checked {  
    CSS-стили  
}
```

Селекторы по тегу, классу, идентификатору

ИМЯ ТЕГА - селектор тега - определяет указанные элементы для форматирования

. - селектор класса - отбирает элементы с заданным классом для форматирования

- селектор идентификатора - отбирает элемент с указанным идентификатором для форматирования

Селектор элемента (тега)

Селекторы элементов позволяют **форматировать все элементы** данного типа **на всех страницах** сайта. Например,

```
h1 {  
    font-family: Lobster, cursive;  
}
```

задаст общий стиль форматирования всех заголовков h1.

Селекторы по тегу, классу, идентификатору

Селектор класса

Селекторы класса позволяют **задавать стили** для одного и более элементов **с одинаковым именем класса**, размещенных **в разных местах** страницы или на разных страницах сайта. Например, для создания заголовка с классом `headline` необходимо добавить атрибут `class` со значением `headline` в открывающий тег `<h1>` и задать стиль для указанного класса. Стили, созданные с помощью класса, можно применять к другим элементам, не обязательно данного типа.

```
.headline {  
  text-transform: uppercase;  
  color: lightblue;  
}
```

CSS

```
<h1 class="headline">Инструкция пользования персональным компьютером</h1>
```

HTML

Если элемент имеет несколько атрибутов класса, их значения объединяются с пробелами.

```
<h1 class="headline post-title">Инструкция пользования персональным компьютером</h1>
```

HTML

Селекторы по тегу, классу, идентификатору

Селектор идентификатора

Селектор идентификатора позволяет **форматировать один конкретный** элемент. Значение `id` должно быть уникальным, на одной странице может встречаться только один раз и должно содержать хотя бы один символ. Значение не должно содержать пробелов.

Нет никаких других ограничений на то, какую форму может принимать `id`, в частности, идентификаторы могут состоять только из цифр, начинаться с цифры, начинаться с подчеркивания, состоять только из знаков препинания и т. д.

Уникальный идентификатор элемента может использоваться для различных целей, в частности, как способ ссылки на конкретные части документа с использованием идентификаторов фрагментов, как способ нацеливания на элемент при создании сценариев и как способ стилизации конкретного элемента из CSS.

```
#sidebar {  
  width: 300px;  
  float: left;  
}
```

CSS

```
<div id="sidebar"></div>
```

HTML

Групповые селекторы

Один и тот же стиль можно одновременно применить к нескольким элементам. Для этого необходимо в левой части объявления перечислить через запятую нужные селекторы:

```
h1,  
h2,  
p,  
span {  
  color: tomato;  
  background: white;  
}
```

CSS

Комбинаторы: потомок()

Селектор потомка — обычно представляется символом **пробела** (" ") — соединяет два селектора так, что элементы, соответствующие второму селектору, выбираются, если они имеют предка (родителя, родителя родителя, родителя родителя родителя и т.д.), соответствующего первому селектору. Селекторы, которые используют комбинатор потомка, называются селекторами потомка.

CSS

Copy

```
body article p
```

E F - селектор потомка - выберет все элементы **F**, расположенные внутри **E**

Комбинаторы: дочерний(>)

Дочерний комбинатор (>) помещается между двумя селекторами CSS. При этом будут выбраны только те элементы, соответствующие второму селектору, которые являются прямыми потомками элементов, соответствующих первому селектору. Все элементы-потомки на более низких уровнях иерархии будут пропущены. Например, чтобы выбрать только те элементы <p>, которые являются дочерними элементами <article>, селектор пишется так:

CSS

Copy

```
article > p
```

Если вы уберёте символ >, указывающий на то, что это селектор с дочерним комбинатором, селектор превратится в селектор всех потомков (комбинатор – пробел)

E > F - дочерний селектор - выберет все элементы **F**, вложенные непосредственно в **E** (первый уровень вложенности)

Комбинаторы: соседний(+)

Соседний родственный селектор (+) используется для выбора элемента, который непосредственно следует за другим элементом и находится на одном с ним уровне иерархии. Например, чтобы выбрать все элементы `<img>`, которые идут сразу после элементов `<p>`:

CSS

Copy

```
p + img
```

Распространённый вариант использования — сделать что-то с абзацем, который следует за заголовком, как в примере ниже. Здесь мы ищем абзац, который непосредственно примыкает к `<h1>`, и стилизуем его.

Если вы вставите какой-то другой элемент, например `<h2>` между `<h1>` и `<p>`, вы обнаружите, что абзац больше не соответствует селектору и поэтому не получает цвет фона и переднего плана, применяемый, когда элемент является соседним.

`E + F` - сестринский селектор - выберет каждый первый элемент `F`, идущий сразу за тегом `E`

Комбинаторы: общий сосед(~)

Если вы хотите выбрать **родственные элементы**, даже если они не являются непосредственными соседями, то вы можете использовать общий родственный комбинатор (`~`). Чтобы выбрать все элементы `<img>`, которые находятся в *любом* месте после элементов `<p>`, надо указать так:

CSS

Copy

```
p ~ img
```

`E ~ F` - сестринский селектор - выберет все элементы `F`, являющиеся сестринскими по отношению к любому `E` (расположенные на одном уровне)

Селекторы атрибутов ([attr], [attr="value"] и др.)

Селекторы атрибутов выбирают элементы на основе имени атрибута или значения атрибута:

- **[атрибут]** — все элементы, содержащие указанный атрибут, **[alt]** — все элементы, для которых задан атрибут alt;
- **селектор[атрибут]** — элементы данного типа, содержащие указанный атрибут, **img[alt]** — только картинки, для которых задан атрибут alt;
- **селектор[атрибут="значение"]** — элементы данного типа, содержащие указанный атрибут с конкретным значением, **img[title="flower"]** — все картинки, название которых содержит слово flower;
- **селектор[атрибут~="значение"]** — элементы частично содержащие данное значение, например, если для элемента задано несколько классов через пробел, **p[class~="feature"]** — абзацы, имя класса которых содержит feature;
- **селектор[атрибут|= "значение"]** — элементы, список значений атрибута которых начинается с указанного слова, **p[class|= "feature"]** — абзацы, имя класса которых feature или начинается на feature;

Селекторы атрибутов ([attr], [attr="value"] и др.)

`[attr]` - элементы данного типа, содержащие указанный атрибут

`[attr="value"]` - элементы, значение атрибута которых равно указанному значению

`[attr ~="value"]` - элементы, значение атрибута которых частично содержит данное значение

`[attr|="value"]` - элементы, список значений атрибута которых начинается с указанного значения

`[attr^="value"]` - элементы, значение атрибута которых начинается с указанного значения

`[attr$="value"]` - элементы, значение атрибута которых заканчивается указанным значением

`[attr*="value"]` - элементы, значение атрибута которых содержит в любом месте указанное значение

:Псевдоклассы

— это селектор, который **выбирает** элементы, находящиеся в **специфическом состоянии**, например, они являются **первым** элементом **своего типа**, или на них **наведён указатель** мыши. Они обычно действуют так, как если бы вы применили класс к какой-то части вашего документа, что часто помогает сократить избыточные классы в разметке и даёт более гибкий, удобный в поддержке код. Например:

CSS Copy

```
div:hover {  
  background-color: #f89b4d;  
}
```

Синтаксис:

```
selector:pseudo-class {  
  property: value;  
}
```

Как и с обычными классами, можно совмещать вместе в одном селекторе любое число псевдоклассов.

:Псевдоклассы

:checked – элементы `<input type="radio">`, `<input type="checkbox">`, а также `<option>`, находящиеся внутри элемента `<select>`

:disabled – заблокированные для выбора и изменения элементы форм

:enabled – не заблокированные для выбора и изменения элементы форм

:indeterminate – элементы радио и чекбокс, находящиеся в неопределённом состоянии

:in-range – поля формы с установленным диапазоном ограничения

:invalid – поля формы, значение которых является недопустимым согласно его типу, указанному в атрибуте `type`

:optional – поля формы, для которых не задан атрибут `required`

:out-of-range – поля формы, имеющие ограничения на ввод значений

:read-only – элементы, для которых задан атрибут `readonly` или `contenteditable`

:required – поля формы, для которых установлен атрибут `required`

:valid – поля формы, введенные значения которых соответствуют типу, указанному в атрибуте `type` или находящиеся в разрешенном диапазоне

:first-child – первый дочерний элемент

:last-child – последний дочерний элемент в родительском контейнере

:nth-last-child() – отбор элементов на основе их индекса, начиная с последнего элемента

:only-child – единственный дочерний элемент

:Псевдоклассы

:nth-of-type() – отбор элементов одного типа на основе их индекса внутри контейнера

:nth-last-of-type() – отбор элементов одного типа на основе их индекса внутри контейнера, начиная с последнего элемента к первому

:first-of-type – первый элемент данного типа среди дочерних элементов родительского элемента-контейнера

:last-of-type – последний элемент данного типа среди дочерних элементов родительского элемента-контейнера

:only-of-type – единственный элемент данного типа в родительском элементе-контейнере

:link – не посещенная ссылка

:visited – посещенная ссылка

:active – элемент, активированный с помощью клика мышки

:focus – применяется, когда элемент имеет фокус – принимает события клавиатуры или мыши или другие формы ввода

:target – элемент с якорем

:lang() – отбор элементов на указанном языке

:not() – отбор элементов, не содержащих указанное значение аргумента

:empty – элемент, не содержащий ни одного дочернего элемента

:root – корневой элемент в документе

:Псевдоклассы

:nth-child() – находит один или более элементов, основываясь на их позиции среди группы соседних элементов

```
/* Выбирает каждый четвёртый элемент  
среди любой группы соседних элементов */  
:nth-child(4n) {  
    color: lime;  
}
```

Синтаксис: указывается с единственным аргументом, описывающим паттерн для выбора элементов.

Ключевые слова:

odd – описывает элементы среди группы соседних с нечётными номерами 1, 3, 5, и т. д.

even – описывает элементы среди группы соседних с чётными номерами 2, 4, 6, и т. д.

Функциональная запись: $\langle An+B \rangle$ – описывает элементы среди группы соседних с номерами, соответствующими паттерну $An+B$ (для каждого целого числа $n \geq 0$). Нумерация элементов начинается с единицы. Значения A и B должны быть $\langle \text{integer} \rangle$ s.

:hover – ссылки и другие элементы, на которые навели курсор мыши

```
:link:hover {  
    outline: dotted red;  
}
```

```
div.menu-bar li:hover > ul {  
    display: block;  
}
```


::Псевдоэлементы

— это ключевое слово, добавляемое к селектору, которое **позволяет стилизовать определённую часть выбранного элемента.**

Псевдоэлементы ведут себя сходным образом, однако они действуют так, как если бы вы добавили в разметку целый новый HTML-элемент, а не применили класс к существующим элементам.

Синтаксис:

```
selector::pseudo-element {  
  property: value;  
}
```

В селекторе можно использовать только один псевдоэлемент. Он должен находиться после простых селекторов в выражении.

Примечание: Как правило, следует использовать двойное двоеточие (::) вместо одинарного (:). В этом состоит различие между псевдоклассами и псевдоэлементами. Однако, так как это различие не присутствовало в старых версиях спецификации W3C, большинство браузеров поддерживают оба синтаксиса для псевдоэлементов.

::Псевдоэлементы

::before – добавление генерируемого содержимого перед элементом

::first-letter – форматирование первой буквы абзаца

::selection – цвет фона при выделении текста на странице с помощью зажатой левой кнопки мыши или другого способа

::after – создание последнего потомка выбранного элемента

::cue – задание титров и других реплик в медиа

::slotted – выбирает фактический элемент, помещённый в слот

::backdrop – подложка, отрисовывается сразу после отрисовки любого элемента в полноэкранном режиме (диалоговое окно открыто с помощью метода `dialog.showModal()`)

::placeholder – используется для стилизации текста-заполнителя в полях ввода

::marker – применение слей к маркеру элемента списка (значок или номер)

::spelling-error – фрагмент текста, который пользовательский агент пометил как содержащий орфографическую ошибку

::grammar-error – сегмент текста, который пользовательский агент пометил как грамматически неверный

::Псевдоэлементы

::first-line – форматирование первой строки абзаца

```
/* Стили для первой строки элемента <p> */  
p::first-line {  
    color: red;  
}
```

```
::first-line {  
    color: blue;  
    text-transform: uppercase;  
  
    /* ПРЕДУПРЕЖДЕНИЕ: НЕ ИСПОЛЬЗУЙТЕ ЭТО */  
    /* Многие свойства запрещены в псевдоэлементах ::first-line */  
    margin-left: 20px;  
    text-indent: 20px;  
}
```

СТИЛИ БУДУТ ПРИМЕНЕНЫ ТОЛЬКО К ПЕРВОЙ СТРОКЕ ЭТОГО ПАРАГРАФА. ПОСЛЕ ЭТОГО,
весь текст будет отображаться как обычно. Увидели, что я имею в виду?

Первая строка этого элемента не будет стилизована специальным образом потому что это не блочный элемент.

Сравнение селекторов (Приоритет стилей)

Наследование и каскад

Наследование и каскад — два фундаментальных понятия в CSS, которые тесно связаны между собой.

Наследование заключается в том, что **элементы наследуют свойства** от своего родителя (элемента, их содержащего).

Каскад проявляется в том, как разные виды таблиц стилей применяются к документу, и как **конфликтующие правила переопределяют друг друга**.

Сравнение селекторов (Приоритет стилей)

Наследование

Наследование является механизмом, с помощью которого **определенные свойства передаются от предка к его потомкам**. Спецификацией CSS предусмотрено наследование свойств, относящихся к текстовому содержимому страницы, таких как color, font, letter-spacing, line-height, list-style, text-align, text-indent, text-transform, visibility, white-space и word-spacing. Во многих случаях это удобно, так как не нужно задавать размер шрифта и семейство шрифтов для каждого элемента веб-страницы.

Свойства, относящиеся к форматированию блоков, не наследуются. Это background, border, display, float и clear, height и width, margin, min-max-height и -width, outline, overflow, padding, position, text-decoration, vertical-align и z-index.

Принудительное наследование

С помощью ключевого слова **inherit** можно **принудить** элемент **наследовать** любое значение свойства родительского элемента. Это работает даже для тех свойств, которые не наследуются по умолчанию.

Сравнение селекторов (Приоритет стилей)

Как задаются и работают CSS-стили

Стили могут наследоваться от родительского элемента (наследуемые свойства или с помощью значения `inherit`). Стили, расположенные в таблице стилей ниже, отменяют стили, расположенные в таблице выше.

К одному элементу могут применяться стили из разных источников. Проверить, какие стили применяются, можно в режиме разработчика браузера. Для этого над элементом нужно щёлкнуть правой кнопкой мыши и выбрать пункт «Посмотреть код» (или что-то аналогичное). В правом столбце будут перечислены все свойства, которые заданы для этого элемента или наследуются от родительского элемента, а также файлы стилей, в которых они указаны, и порядковый номер строки кода.

При определении стиля можно использовать любую комбинацию селекторов — селектор элемента, псевдокласса элемента, класса или идентификатора элемента.

```
div {border: 1px solid #eee;}  
#wrap {width: 500px;}  
.box {float: left;}  
.clear {clear: both;}
```

CSS

```
<div id="wrap" class="box clear"></div>
```

HTML

Сравнение селекторов (Приоритет стилей)

Каскад

Каскадирование — это механизм, который **управляет конечным результатом** в ситуации, когда **к одному элементу применяются разные CSS-правила**. Существует три критерия, которые определяют порядок применения свойств — правило !important, специфичность и порядок, в котором подключены таблицы стилей.

Правило !important

Вес правила можно задать с помощью ключевого слова !important, которое добавляется сразу после значения свойства. Правило необходимо размещать в конец объявления перед закрывающей скобкой, без пробела. Такое объявление будет иметь **приоритет над всеми остальными правилами**. Это правило позволяет отменить значение свойства и установить новое для элемента из группы элементов в случае, когда нет прямого доступа к файлу со стилями.

Сравнение селекторов (Приоритет стилей)

Как работает !important:

Превосходство над другими стилями. Правило с !important **переопределяет все предыдущие стили** для того же свойства на элементе, даже если они имеют более высокую специфичность. Это означает, что значение, указанное с !important, будет иметь приоритет над любыми другими значениями для того же свойства.

Синтаксис. Модификатор добавляется в конце значения свойства, через пробел, перед точкой с запятой:

```
selector {  
    property: value !important;  
}
```

Влияние на каскад. !important **«сбрасывает» каскад**, то есть порядок определения приоритета стилей. Если к одному элементу применяется несколько правил с !important, то начинает действовать специфичность селекторов: выигрывает правило с более конкретным селектором.

Сравнение селекторов (Приоритет стилей)

Важные нюансы:

Специфичность селекторов. Сам по себе **!important** не влияет на специфичность селекторов, но может **влиять на каскад**, если к элементу применяется несколько правил с **!important**.

*Использование **!important** требует осторожности.* Его частое применение может привести к сложностям в поддержке кода, конфликтам стилей и снижению читаемости кода. Рекомендуется использовать **!important** только в крайних случаях, когда другие методы не работают.

*Некоторые ситуации, когда **!important** может быть полезен:* переопределение встроенных стилей, стилей сторонних библиотек или фреймворков, работа с элементами, которые не подлежат изменению (например, в некоторых системах), тестирование и отладка.

Пример: если есть два правила для элемента, одно без **!important**, а другое с ним, то победит правило с **!important**. Если же есть два правила с **!important**, но с разными селекторами, то победит правило с более специфичным селектором.

Сравнение селекторов (Приоритет стилей)

Как именно сравниваются селекторы с **!important** (алгоритм из 8 разрядов):

Алгоритм «взвешивания»:

1. Смотрим на крайние левые числа
2. Выбираем из них большее. — именно этот селектор и будет самым тяжелым
3. Если числа одинаковые, сдвигаемся на число вправо и повторяем инструкции из пункта 2.
4. Если все числа одинаковые то применяются стили из селектора который был объявлен последним

Итак, взвешиваем — сначала представим пару рядов из 8 чисел:

```
0,1,0,0,0,0,0,0
```

```
1,0,0,0,0,0,0,0
```

Сравнение селекторов (Приоритет стилей)

Как собственно превратить обычный селектор в такие понятные и красивые цифры?

- ❑ **Теги.** за каждый тег в селекторе можно накинуть в самое правое число единичку:

```
a – это 0,0,0,0,0,0,0,1  
div a – это 0,0,0,0,0,0,0,2
```

- ❑ **Классы,** за каждый класс или псевдокласс в селекторе можно накинуть по единичке во второе число справа

```
.head .logo – это 0,0,0,0,0,0,2,0  
.logo.big – 0,0,0,0,0,0,2,0  
div:first-child – 0,0,0,0,0,0,1,1  
.logog > .big – и это тоже 0,0,0,0,0,0,2,0
```

- ❑ За каждый **ID** в селекторе добавляем по единичке в третье справа число.

```
#head – это 0,0,0,0,0,0,1,0,0  
#head #logo – тоже 0,0,0,0,0,0,2,0,0
```

Сравнение селекторов (Приоритет стилей)

Каковы всё таки размеры?

- Селектор ***** абсолютно невесомый, то есть совсем.
- **Селектор атрибутов** это самый обычный псевдокласс и весит он столько же сколько и обычные классы
- **Любой инлайновый стиль** написанный в атрибуте `style=""` элемента автоматически получает приоритет **0,0,0,0,1,0,0,0**, что сразу делает его очень крутым.
- А следующие четыре цифры это все наши старые знакомые только с атрибутом **!important**

```
div
{
  background-color: gray !important;
}
```

Имеет вес при определении свойства background-color - 0,0,0,1,0,0,0,0

```
.header
{
  background-color: gray !important;
}
```

0,0,1,0,0,0,0,0

Сравнение селекторов (Приоритет стилей)

Специфичность

Для каждого правила браузер вычисляет специфичность селектора, и если у элемента имеются конфликтующие объявления свойств, во внимание принимается **правило, имеющее наибольшую специфичность**. Значение специфичности состоит из четырех частей: 0, 0, 0, 0. Специфичность селектора определяется следующим образом:

- для id добавляется 0, 1, 0, 0;
- для class, атрибута и псевдокласса добавляется 0, 0, 1, 0;
- для каждого элемента и псевдоэлемента добавляется 0, 0, 0, 1;
- для встроенного стиля, добавленного непосредственно к элементу — 1, 0, 0, 0;
- универсальный селектор не имеет специфичности.

Сравнение селекторов (Приоритет стилей)

```
h1 {color: lightblue;} /*специфичность 0, 0, 0, 1*/
em {color: silver;} /*специфичность 0, 0, 0, 1*/
h1 em {color: gold;} /*специфичность: 0, 0, 0, 1 + 0, 0, 0, 1 = 0, 0,
0, 2*/
div#main p.about {color: blue;} /*специфичность: 0, 0, 0, 1 + 0, 1, 0,
0 + 0, 0, 1 + 0, 0, 1, 0 = 0, 1, 1, 2*/
.sidebar {color: grey;} /*специфичность 0, 0, 1, 0*/
#sidebar {color: orange;} /*специфичность 0, 1, 0, 0*/
li#sidebar {color: aqua;} /*специфичность: 0, 0, 0, 1 + 0, 1, 0, 0 =
0, 1, 0, 1*/
```

В результате к элементу применятся те правила, специфичность которых больше. Например, если на элемент действуют две специфичности со значениями 0, 0, 0, 2 и 0, 1, 0, 1, то выиграет второе правило.

Порядок подключённых таблиц

Вы можете создать несколько внешних таблиц стилей и подключить их к одной веб-странице. Если в разных таблицах будут встречаться разные значения свойств одного элемента, то в результате к элементу применится правило, находящееся в таблице стилей, идущей в списке ниже.

CSS-переменные (кастомные свойства) : --var, var()

Имена свойств с префиксом `--`, представляют собой *кастомные свойства*, содержащие значения, которые можно переиспользовать в прочих декларациях, при помощи функции (`var()`).

Область действия кастомного свойства ограничена элементом, для которого оно объявлено.

Кастомные свойства также могут менять свои значения согласно правилам каскадирования.

Синтаксис:

CSS

Copy

```
--somekeyword: left;  
--somecolor: #0000ff;  
--somecomplexvalue: 3px 6px rgb(20, 32, 54);
```

Основное использование:

CSS

Объявление переменной

```
element {  
  --main-bg-color: brown;  
}
```

CSS

Использование переменной

Copy

```
element {  
  background-color: var(--main-bg-color);  
}
```

Примечание: Имена кастомных свойств чувствительны к регистру — таким образом `--my-color` и `-My-color` будут определены, как два различных кастомных свойства.

CSS-переменные (кастомные свойства) : --var, var()

CSS-переменные (custom properties) определяются внутри селекторов, ограничивая область видимости (scope) этим элементом и его потомками. Использование **:root** делает переменные **глобальными**, доступными всему документу. Локальные переменные удобны для компонентов, а глобальные — для тем (цвета, шрифты).

Ключевые аспекты:

Локальная область видимости: Переменная работает только внутри выбранного элемента (например, `.card { --main-color: blue; }`).

Глобальная область видимости (:root): Переменная, определенная в `:root`, доступна во всем CSSOM.

Синтаксис: Объявление через `--name`, использование через `var(--name)`.

CSS

```
element {  
  --main-bg-color: brown;  
}
```

CSS

```
element {  
  background-color: var(--main-bg-color);  
}
```